

PER2-PROTAC: QC PSM-level quantification and summarisation to protein-level abundance

Tom Smith

2026-08-24

Contents

1 Purpose	2
2 Setup	3
2.1 Defining the contaminant proteins	3
2.2 Define the experiment design	4
3 Read in input data	4
3.1 Creating the QFeatures object	4
4 Filtering PSMs	6
4.1 Phosphosite annotation	7
4.2 Removing PSMs with high co-isolation	8
4.3 Normalise	9
5 Summarising the total PSM abundances to higher level abundances	10
6 Inspecting the number of peptides and proteins through the processing steps	14
6.1 Samples per PSM	14
6.2 Samples per protein	16
7 Saving out results	19
8 Normalising phosphoproteomics to protein abundances	21
9 Session info	22

1 Purpose

This notebook reads the PSM-level output from the MaxQuant search of the PER2-PROTAC experiment (a single TMT18-plex, DDA acquisition, run as separate total-proteome and phospho-enriched fractionated experiments sharing the same 18 reporter channels), applies routine PSM quality filtering, and summarises the quantification to protein-level and phosphosite-level abundances. The result is normalised protein and phosphopeptide abundance matrices ready for statistical testing in notebook 4.

The TMT18 plex multiplexes samples from more than one experiment: alongside the PER2-PROTAC (P2H) samples relevant to this project, it also carries P1H and combined P1H/P2H channels belonging to a different, unrelated experiment. This notebook reads in all 18 channels but subsets to just the 6 P2H channels (DMSO x3, PROTAC x3) immediately after loading.

The total and phospho data are processed through the same sequence of steps, so throughout the notebook the two experiments are held together in a list of two `QFeatures` objects (`qfs$total` and `qfs$phospho`) and looped over. Where the phospho experiment needs additional handling (PTM localisation, PSM-to-phosphosite summarisation) this is called out explicitly.

Input:

- `raw/PROTAC/evidence.txt.gz` — MaxQuant PSM-level output, covering both the total-proteome and phospho-enriched (fractionated) runs, distinguished by whether the `Raw.file` name contains `phos`.
- `raw/PROTAC/proteinGroups.txt.gz` — protein-level search results, used here for the `Q.value` (FDR) column.
- `raw/PROTAC/exp_design.xlsx` — the TMT channel / genotype / condition / replicate layout.
- `external/uniprot2details_unique.rds` from notebook 0 — UniProt protein annotations.

Output:

- `notebooks/output/PROTAC_qfs.rds` — the list of two `QFeatures` objects (total and phospho) retaining every processing stage, consumed by notebook 4.
- `results/PROTAC_wide_format_protein.tsv` — bridge-free, normalised protein abundances in wide (feature x sample) format.
- `results/PROTAC_long_format_protein.tsv` and `results/PROTAC_long_format_phospho.tsv` (with matching `.rds` files in `notebooks/output/`) — the protein and phosphopeptide abundances in tidy long format.
- `results/PROTAC_long_format_phosphopeptide_norm_to_protein.tsv` (and matching `.rds`) — phosphopeptide abundances joined to their parent protein abundance, with a phospho-normalised-to-protein column.
- `notebooks/output/PROTAC_long_long_format_phosphopeptide_norm_to_protein.rds` — the same data with protein / phosphopeptide / normalised-phosphopeptide values held as separate rows rather than columns, used for some of the exploratory plots in notebook 4.

Analysis:

1. **Read data** — read the PSM-level evidence file, join in protein annotations and the protein-level FDR (`Q.value`), split into total-proteome and phospho-enriched PSMs by raw file name, build one `QFeatures` object per experiment, and subset both to the PER2-PROTAC (P2H) channels.
2. **Generic filtering** — remove contaminant PSMs and PSMs lacking a unique master protein; for the total-proteome data, additionally remove PSMs belonging to proteins that fail the 1% FDR threshold.
3. **Phospho-specific filtering** — determine phosphosite localisation from the MaxQuant PTM probabilities, retain only confidently localised phosphosites, and record the phosphosite position(s) and surrounding sequence on the protein.

4. **Co-isolation filtering** — remove PSMs where more than 50% of the isolated MS1 signal came from co-isolated (non-target) species, which would otherwise compress the TMT reporter ratios.
5. **Normalisation** — median-normalise the PSM abundances (both experiments independently, since there is a single plex here and so no inter-plex batch effect to correct for).
6. **Summarisation** — after removing PSMs with more than one missing value and imputing the small remainder, sum PSM intensities to protein level (total) or phosphosite level (phospho) and log2-transform.
7. **Export** — save the `QFeatures` object and the wide/long abundance tables, including phosphopeptide abundances normalised to their parent protein.

How to read the results: the key objects for downstream notebooks are the long-format tables. `long_format_protein` / `long_format_phospho` give the normalised abundance of each protein / phosphopeptide in each sample. `long_format_phosphopeptide_norm_to_protein` additionally carries, for each phosphopeptide, the abundance of its parent protein and the difference between them (`abundance_phosphopeptide_prot_norm`), which is the quantity used in notebook 4 to detect phosphorylation changes that are not simply driven by changes in protein abundance.

2 Setup

Load required packages

```
library(biomasslmb)
library(ggplot2)
library(tidyr)
library(dplyr)
library(here)
library(QFeatures)
```

```
## Warning: package 'MultiAssayExperiment' was built under R version 4.5.3
```

```
## Warning: package 'S4Vectors' was built under R version 4.5.3
```

```
library(openxlsx)
```

Read in Uniprot protein details

```
uniprot2details_unique <- readRDS(here('external/uniprot2details_unique.rds'))
```

2.1 Defining the contaminant proteins

We need to remove contaminant proteins. Below, we parse the MaxQuant contaminants fasta to extract the IDs for the proteins.

```
contaminant_proteins <- get_maxquant_cont_accessions()
```

```
## No contaminants fasta supplied, defaulting to contaminants.fasta.gz from biomasslmb
```

2.2 Define the experiment design

We can extract the details for the experiment to sample mapping from the Excel file provided. The `sample_conditions` field encodes genotype, condition, and replicate as a single space-separated string (e.g. "P2H DMSO 1"), which we split into separate columns below; `reporter_n` is the TMT channel used to join this table to the PSM quantification data.

```
exp_design <- openxlsx::read.xlsx(here('raw/PROTAC/exp_design.xlsx'),
                                startPos = 7, colNames = FALSE) %>%
  setNames(c('sample_name', 'sample_conditions', 'reporter_n',
            'tag', 'reporter_mass')) %>%
  separate(sample_conditions, into=c('genotype', 'condition', 'replicate'),
            sep=' ', remove = FALSE) %>%
  mutate(genotype=ifelse(genotype=='P1H/P2H', 'P1H.P2H', genotype)) %>%
  mutate(quantCols=reporter_n)
```

3 Read in input data

We start by reading in quantitative proteomics data into a `QFeatures` object, which is the standard Bioconductor object for holding quantitative proteomics data. See here for documentation about the `QFeatures` object.

```
pg_inf <- here('raw/PROTAC/proteinGroups.txt.gz')
pg_df <- read.delim(pg_inf)
table(pg_df$Q.value<0.01)
```

```
##
## FALSE TRUE
## 548 8847
```

3.1 Creating the QFeatures object

Now we can now create the `QFeatures` object. Here, we will create two PSM-level `QFeatures` objects, one for the total proteome and one for the phosphoproteome. Both experiments were acquired with offline fractionation (multiple raw files per experiment, all using the same 18 TMT reporter channels), so all fractions for an experiment are read into a single assay.

```
psm_inf <- here('raw/PROTAC/evidence.txt.gz')

infdf <- read.delim(psm_inf)

# Merge on the leading razor protein column from the Peptides file
infdf <- merge(infdf, uniprot2details_unique,
              by.x='Leading.razor.protein', by.y='UniprotID', all.x=TRUE) %>%
  merge(pg_df[,c('Majority.protein.IDs', 'Q.value')], 'Leading.razor.protein',
        by.y='Majority.protein.IDs', all.x=TRUE)

# Define the abundance columns
abundance_cols <- 56:73
```

```

# Rename the abundance columns to be just the reporter mass
colnames(infdf)[abundance_cols] <- gsub(
  'Reporter.intensity.corrected.', '', colnames(infdf)[abundance_cols])

# Split into phospho and total
phospho_infdf <- filter(infdf, grepl('phos', Raw.file))
total_infdf <- filter(infdf, !grepl('phos', Raw.file))

# Create QFeatures object for total
tmt_qf <- QFeatures::readQFeatures(
  assayData = total_infdf,
  quantCols = abundance_cols,
  colData = exp_design, name='psm_raw')

```

```

## Checking arguments.

## Loading data as a 'SummarizedExperiment' object.

## Formatting sample annotations (colData).

## Formatting data as a 'QFeatures' object.

```

```

# Copy over the colData
colData(tmt_qf[['psm_raw']]) <- colData(tmt_qf)

# Create QFeatures object for phospho
phospho_tmt_qf <- QFeatures::readQFeatures(
  assayData = phospho_infdf,
  quantCols = abundance_cols,
  colData = exp_design, name='psm_raw')

```

```

## Checking arguments.

## Loading data as a 'SummarizedExperiment' object.

## Formatting sample annotations (colData).

## Formatting data as a 'QFeatures' object.

```

```

colData(phospho_tmt_qf[['psm_raw']]) <- colData(phospho_tmt_qf)

# Combine into a list and replace zeroes with NAs
qfs <- list('total'=tmt_qf, 'phospho'=phospho_tmt_qf)
qfs <- lapply(qfs, function(x) zeroIsNA(x, names(x)))

```

As noted above, the TMT18 plex multiplexes samples from more than one experiment. Below, we subset both `QFeatures` objects down to just the 6 P2H (PER2-PROTAC) channels — 3 DMSO and 3 PROTAC replicates — that are relevant to this project.

```
qfs <- lapply(qfs, function(x) x[,colData(x)$genotype == 'P2H'])
```

4 Filtering PSMs

We first perform routine filtering to remove PSMs that:

- Could originate from contaminants. See `?filter_features` for further details, including the removal of ‘associated cRAP’.
- Don’t have a unique master protein.

```
for(qf in names(qfs)){  
  
  in_assay_name <- 'psm_raw'  
  out_assay_name <- 'psm_filtered'  
  
  qfs[[qf]][[out_assay_name]] <- filter_features_mq_dda(  
    qfs[[qf]][[in_assay_name]],  
    contaminant_proteins=contaminant_proteins,  
    filter_contaminant=TRUE,  
    filter_associated_contaminant=TRUE,  
    unique_master=TRUE)  
  
  if(qf == 'total'){  
    qfs[[qf]] <- filterFeatures(qfs[[qf]], i=out_assay_name, ~ Q.value <= 0.01)  
  
    biomasslmb:::message_parse(rowData(qfs[[qf]][[out_assay_name]]),  
                              "Leading.razor.protein",  
                              "Proteins with Q-Value > 0.01 removed")  
  }  
}
```

```
## Filtering data...
```

```
## 113093 features found from 8177 master proteins => Input
```

```
## 112923 features found from 8060 master proteins => Removed hits to decoy database
```

```
## 246 contaminant proteins supplied
```

```
## 1318 proteins identified as 'contaminant associated'
```

```
## 111841 features found from 7994 master proteins => contaminant features removed
```

```
## 110711 features found from 7962 master proteins => associated contaminant features removed
```

```
## 110711 features found from 7962 master proteins => MaxQuant-labelled 'Contaminants' removed
```

```
## 110711 features found from 7962 master proteins => features without a master protein removed
```

```

## 106115 features found from 7940 master proteins => features with non-unique master proteins removed

## 85996 features found from 7806 master proteins => features without quantification removed

## 'Q.value' found in 2 out of 2 assay(s).

## 85931 features found from 7748 master proteins => Proteins with Q-Value > 0.01 removed

## Filtering data...

## 64807 features found from 6814 master proteins => Input

## 64532 features found from 6611 master proteins => Removed hits to decoy database

## 246 contaminant proteins supplied

## 646 proteins identified as 'contaminant associated'

## 64074 features found from 6582 master proteins => contaminant features removed

## 63468 features found from 6563 master proteins => associated contaminant features removed

## 63468 features found from 6563 master proteins => MaxQuant-labelled 'Contaminants' removed

## 63468 features found from 6563 master proteins => features without a master protein removed

## 62013 features found from 6544 master proteins => features with non-unique master proteins removed

## 49175 features found from 6095 master proteins => features without quantification removed

```

4.1 Phosphosite annotation

The phospho data needs additional processing to ensure each phosphosite is confidently localised, and to determine the position of the phosphosite(s) on the protein. Below, we filter to PSMs with a confidently localised phosphosite (probability > 0.5), determine the phosphosite position(s) using a temporary reference proteome FASTA built from the detected proteins, and build a `prot_phos` identifier combining the protein accession with the phosphosite name — this is the unit we later summarise to (a specific phosphosite, or combination of sites, on a specific protein). We also record the 15-amino-acid sequence surrounding each phosphosite, used later for reporting.

```

proteome_fasta_filepath <- tempfile()
make_fasta(unique(phospho_infd$Leading.razor.protein), file=proteome_fasta_filepath)

```

```

## Downloading from UniProtKB release: 2026_02

```

```

in_assay_name <- 'psm_filtered'
out_assay_name <- 'psm_annot'

qfs$phospho[[out_assay_name]] <- biomasslmb::filter_maxquant_ptm(
  qfs$phospho, in_assay_name)

# Retain phosphosites in all contexts (ptms_to_retain=c('p', '(ph)', '(ac)p'))
qfs$phospho[[out_assay_name]] <- add_filter_ptm_pos_rowdata_mq(
  qfs$phospho[[out_assay_name]], verbose = TRUE,
  min_prob = 0.501, filter_pep_by_prob = TRUE)

```

The following PTMs were detected: Phospho (STY): 50438; Oxidation (M): 6067; Acetyl (Protein N-term)

```

# Add phospho site positions
qfs$phospho[[out_assay_name]] <- add_ptm_positions(
  obj=qfs$phospho[[out_assay_name]], proteome_fasta=proteome_fasta_filepath)

# Create a combined protein_phosphosite identifier
rowData(qfs$phospho)[[out_assay_name]]$prot_phos <- paste(
  rowData(qfs$phospho)[[out_assay_name]]$Leading.razor.protein,
  gsub(' ', '', rowData(qfs$phospho)[[out_assay_name]]$ptm_name), sep='_')

# Add phospho site sequence
qfs$phospho[[out_assay_name]] <- qfs$phospho[[out_assay_name]] %>%
  add_site_sequence(proteome_fasta = proteome_fasta_filepath,
                    master_protein_col = 'Leading.razor.protein',
                    sequence_pad=7)

# Clean up the fasta file now we've finished with it
file.remove(proteome_fasta_filepath)

```

```
## [1] TRUE
```

4.2 Removing PSMs with high co-isolation

Co-isolation of other peptides within the MS1 isolation window compresses TMT ratios. MaxQuant reports the Parent Ion Fraction (PIF), the proportion of the ion current in the isolation window belonging to the target peptide. Below, we remove PSMs where more than 50% of the isolated signal came from other species.

Note that PIF is NA for PSMs where no MS1 isotope pattern was assembled (Type=='MSMS'), so that purity could not be assessed. These are retained, since we have no evidence that they are co-isolated.

```

for(qf in names(qfs)){

  if(qf=='total'){
    in_assay_name <- 'psm_filtered'
  } else{
    in_assay_name <- 'psm_annot'
  }

  out_assay_name <- 'psm_filtered_interference'

```

```

pif <- rowData(qfs[[qf]][[in_assay_name]])$PIF

qfs[[qf]][[out_assay_name]] <- qfs[[qf]][[in_assay_name]][is.na(pif)|pif>=0.5,]

biomasslmb::message_parse(rowData(qfs[[qf]][[out_assay_name]]),
                          "Leading.razor.protein",
                          "Removing PSMs with co-isolation > 50%")
}

```

```
## 82715 features found from 7666 master proteins => Removing PSMs with co-isolation > 50%
```

```
## 37355 features found from 5263 master proteins => Removing PSMs with co-isolation > 50%
```

4.3 Normalise

Next, we normalise the PSM level intensities by performing diff.median normalisation. Since all samples for a given experiment are quantified within a single TMT18 plex, there is no inter-plex batch effect to correct for here (unlike a multi-plex TMT design), so a single round of median normalisation per experiment is sufficient. We log2-transform first so the normalisation behaves as expected, then exponentiate back to the original scale, since the downstream PSM-to-feature summarisation sums non-transformed intensities.

```

for(qf in names(qfs)){

  in_assay_name <- 'psm_filtered_interference'

  out_assay_name <- 'psm_filtered_norm'

  # Log2-transform, normalise, then back-transform
  qfs[[qf]][[out_assay_name]] <- QFeatures::logTransform(
    qfs[[qf]][[in_assay_name]], base=2)
  qfs[[qf]][[out_assay_name]] <- QFeatures::normalize(
    qfs[[qf]][[out_assay_name]], method='diff.median')

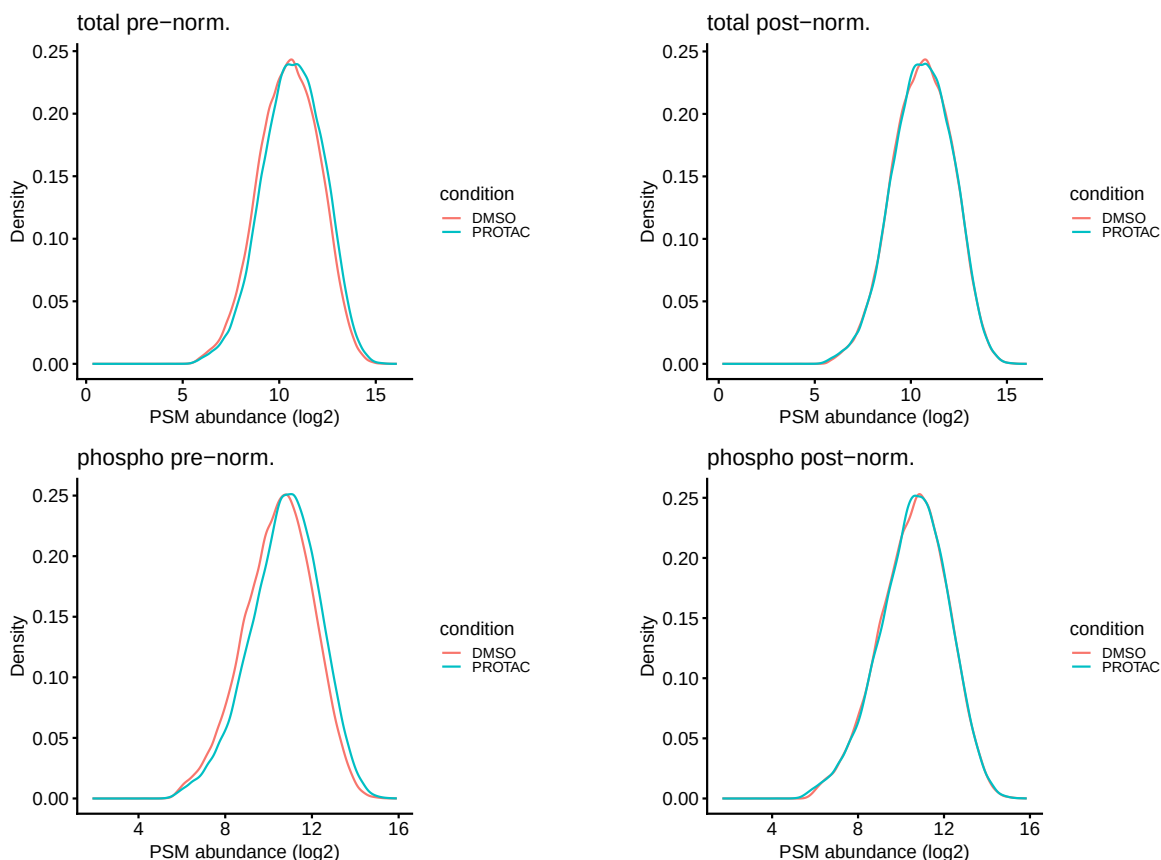
  assay(qfs[[qf]][[out_assay_name]]) <- 2^assay(qfs[[qf]][[out_assay_name]])

  # Plot the peptide-level quantification distributions per sample
  p <- plot_quant(qfs[[qf]][[in_assay_name]], log2transform=TRUE, method='density') +
    theme_biomasslmb() +
    xlab('PSM abundance (log2)') +
    theme(legend.key.height=unit(3, 'mm')) +
    aes(colour=condition) +
    ggtitle(paste0(qf, ' pre-norm.'))
  print(p)

  p <- plot_quant(qfs[[qf]][[out_assay_name]], log2transform=TRUE, method='density') +
    theme_biomasslmb() +
    xlab('PSM abundance (log2)') +
    theme(legend.key.height=unit(3, 'mm')) +
    aes(colour=condition) +
    ggtitle(paste0(qf, ' post-norm.'))
  print(p)
}

```

}

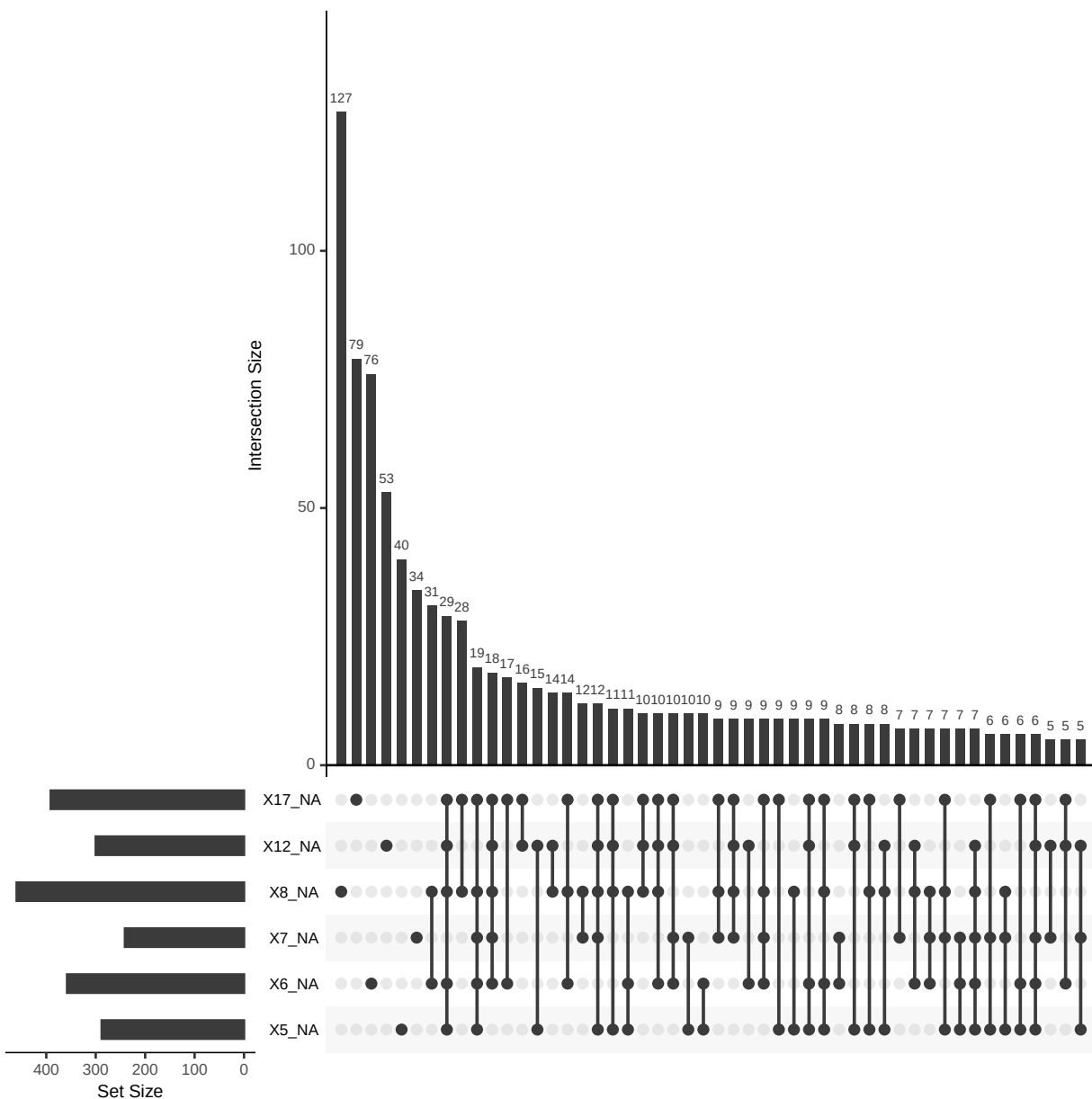


5 Summarising the total PSM abundances to higher level abundances

Now that we have inspected the PSM-level quantification and filtered the PSMs, we can summarise the total PSMs to protein-level abundances and the phospho PSMs to phosphosite-level abundance

For PSM to protein summarisation with TMT quantification, simply summing together the PSM-level abundances provides accurate estimates, so long as there are no missing values. Summing has the benefit of weighting the summarisation towards the most abundant PSMs, which are more likely to be accurately quantified. Where many missing values are present, the `MsCoreUtils::robustSummary` method should be preferred, since it is able to summarise accurately, even in the presence of missing values. Here, we have very few missing values, so we will use summation, with a prior step to impute the missing values.

```
plot_missing_upset(obj = qfs$total, i='psm_filtered_norm')
```



```
nNA(qfs$total, i=names(qfs$total)[grepl('_norm', names(qfs$total))])$nNA %>%
  data.frame() %>% knitr::kable()
```

assay	nNA	pNA
psm_filtered_norm	2038	0.0041065

```
nNA(qfs$phospho, i=names(qfs$phospho)[grepl('_norm', names(qfs$phospho))])$nNA %>%
  data.frame() %>% knitr::kable()
```

assay	nNA	pNA
psm_filtered_norm	1702	0.0075938

Below, we remove PSMs with more than one missing value (out of 6 samples), then impute the small remaining fraction of missing values using `MinProb` (which draws imputed values from the low-abundance tail of the observed distribution, appropriate for values missing because they fell below the limit of detection), so that every retained PSM has a complete set of 6 abundance values ready for summation. A fixed random seed is set beforehand since the imputation involves random draws.

```
set.seed(42)

for(qf in names(qfs)){

  in_assay_name <- 'psm_filtered_norm'
  out_assay_name <- 'psm_filtered_forSum'

  # Remove PSMs with > 1 missing values
  qfs[[qf]][[out_assay_name]] <- QFeatures::filterNA(
    qfs[[qf]][[in_assay_name]], 1/6)

  biomasslmb::message_parse(rowData(qfs$total[[out_assay_name]]),
    'Leading.razor.protein',
    "Removing PSMs with excessive missing values")

  # Check remaining missing values
  print(sprintf('Missing values in %s = %s %%',
    qf,
    round(100*mean(is.na(assay(qfs[[qf]][[out_assay_name]]))), 3)))

  # Impute remaining missing values using MinProb. We log2-transform, impute, then back-transform
  qfs[[qf]][[out_assay_name]] <- logTransform(
    qfs[[qf]][[out_assay_name]], base=2)
  qfs[[qf]][[out_assay_name]] <- QFeatures::impute(
    qfs[[qf]][[out_assay_name]], method='MinProb')
  assay(qfs[[qf]][[out_assay_name]]) <- 2^assay(qfs[[qf]][[out_assay_name]])

}
```

```
## 82198 features found from 7648 master proteins => Removing PSMs with excessive missing values
```

```
## [1] "Missing values in total = 0.083 %"
```

```
## Loading required namespace: imputeLCMD
```

```
## Imputing along margin 2 (samples/columns).
```

```
## [1] 0.1814955
```

```
## 82198 features found from 7648 master proteins => Removing PSMs with excessive missing values
```

```
## [1] "Missing values in phospho = 0.144 %"

## Imputing along margin 2 (samples/columns).

## [1] 0.2113503
```

Finally, we perform the summarisation: total PSMs are summed to `Leading.razor.protein` (protein level), and phospho PSMs are summed to `prot_phos` (phosphosite level).

```
for(qf in names(qfs)){

  in_assay_name <- 'psm_filtered_forSum'

  if(qf=='total'){
    out_assay_name <- 'protein'
    group_col <- "Leading.razor.protein"
  } else{
    out_assay_name<- 'phosphopeptide'
    group_col <- "prot_phos"
  }

  qfs[[qf]] <- aggregateFeatures(qfs[[qf]],
                                i = in_assay_name,
                                fcol = group_col,
                                name = out_assay_name,
                                fun = base::colSums)

  # Log2-transform the summarised abundances for downstream analysis
  qfs[[qf]][[out_assay_name]] <- logTransform(qfs[[qf]][[out_assay_name]], base=2)

  qfs[[qf]][[out_assay_name]]
}
```

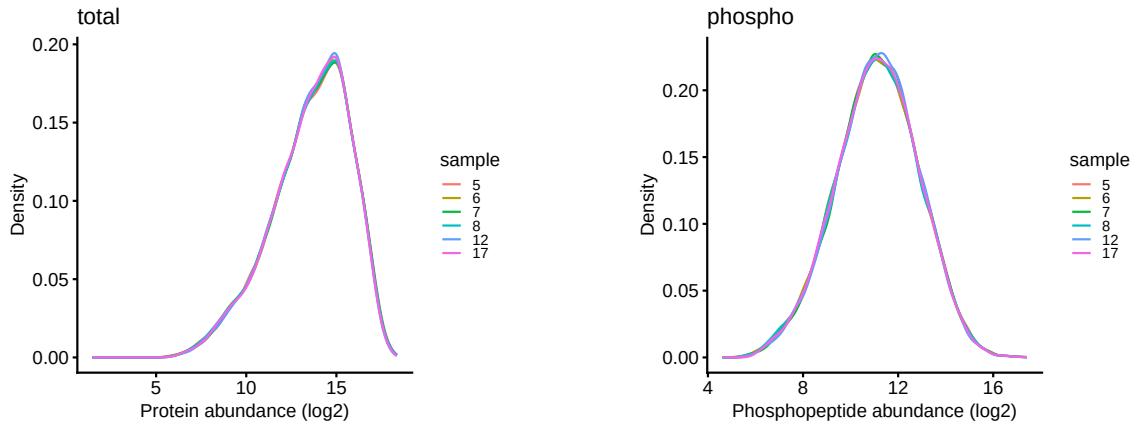
A final check that that the summarised abundances look sensible

```
for(qf in names(qfs)){

  if(qf=='total'){
    assay_name <- 'protein'
    feature <- 'Protein'
  } else{
    assay_name <- 'phosphopeptide'
    feature <- 'Phosphopeptide'
  }

  p <- plot_quant(qfs[[qf]][[assay_name]], log2transform=FALSE, method='density') +
    theme_biomasslmb() +
    xlab(sprintf('%s abundance (log2)', feature)) +
    theme(legend.key.height=unit(3, 'mm')) +
    ggtitle(qf)

  print(p)
}
```



6 Inspecting the number of peptides and proteins through the processing steps

Now that we have performed all the filtering steps, it's helpful to visualise how many peptides/proteins were retained at each level of the processing.

6.1 Samples per PSM

We'll start by inspecting the number of peptides in each experiment. We set the row variables to be `Sequence`, `Modifications` and `Retention.time` so that we count the number of unique PSMs.

```
for(qf in names(qfs)){

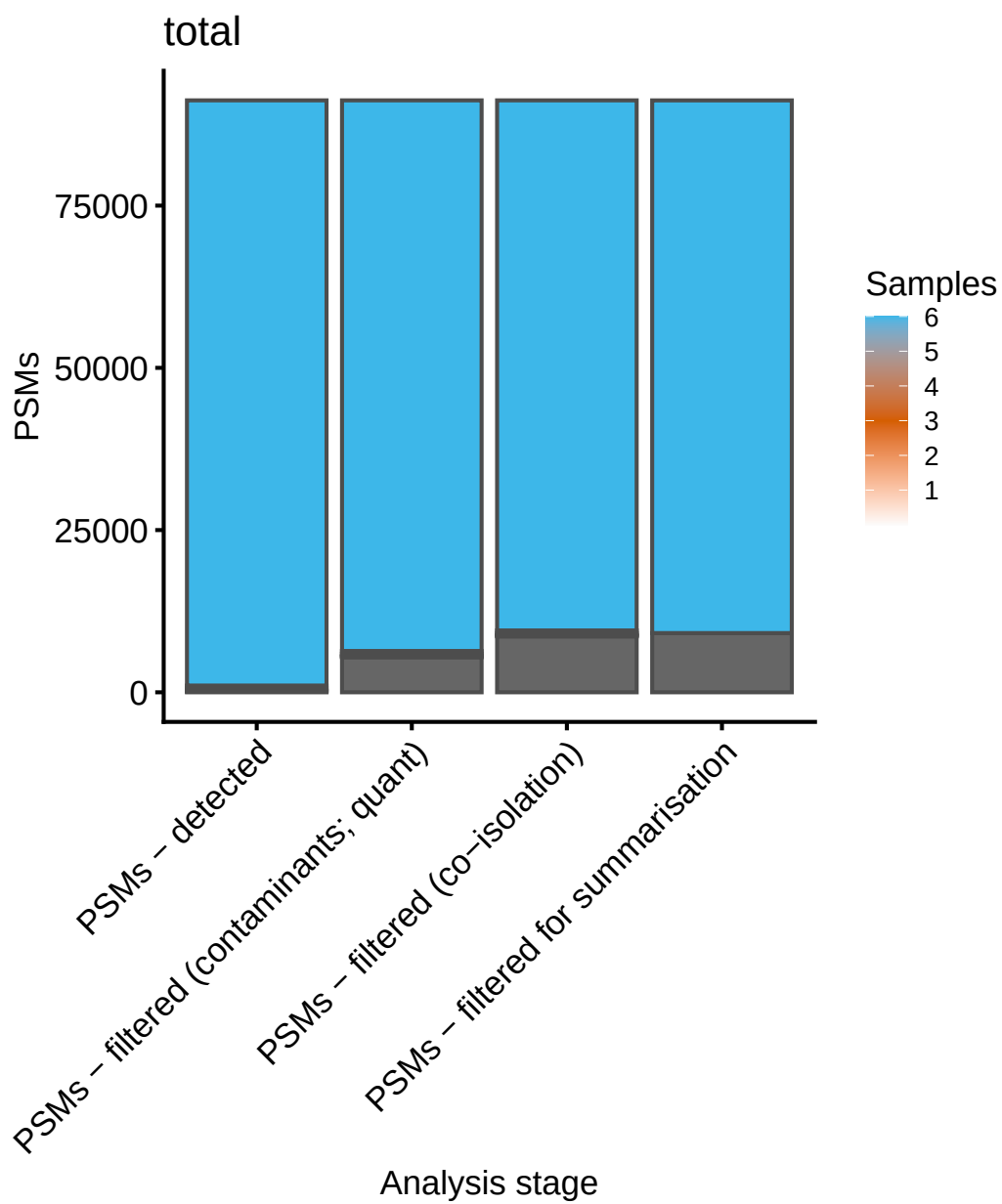
  rowvars <- c('Sequence', 'Modifications', 'Retention.time')

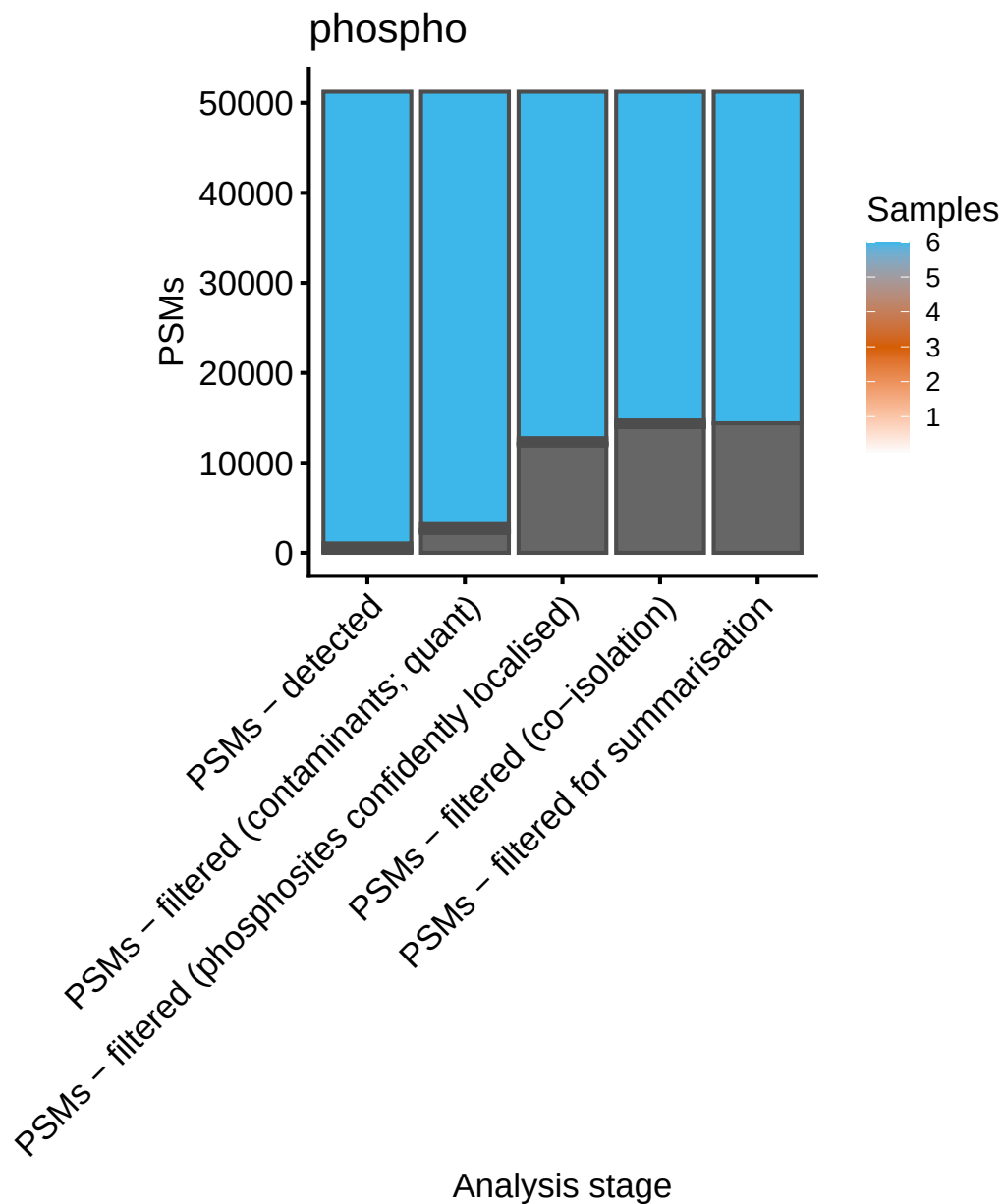
  if(qf=='total'){
    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered (co-isolation)' = 'psm_filtered_interference',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum')
  } else{
    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered (phosphosites confidently localised)' = 'psm_annot',
      'PSMs - filtered (co-isolation)' = 'psm_filtered_interference',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum')
  }

  samples_present <- get_samples_present(qfs[[qf]][, , unname(rename_cols)],
                                         rowvars, rename_cols)

  p <- plot_samples_present(samples_present, rowvars, breaks=seq(1,6,1)) +
    ylab('PSMs') + ggtitle(qf)
```

```
print(p)  
}
```





6.2 Samples per protein

We need to set the row variables to be just the `Leading.razor.protein` column so we count the number of unique proteins.

```
for(qf in names(qfs)){
  print(qf)

  rowvars <- c('Leading.razor.protein')

  if(qf=='total'){
    feature <- 'Proteins'
```

```

    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered (co-isolation)' = 'psm_filtered_interference',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum',
      'Protein' = 'protein')
  } else{
    feature <- 'Phosphopeptides'

    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered (phosphosites confidently localised)' = 'psm_annot',
      'PSMs - filtered (co-isolation)' = 'psm_filtered_interference',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum',
      'Phosphopeptide' = 'phosphopeptide')

  }

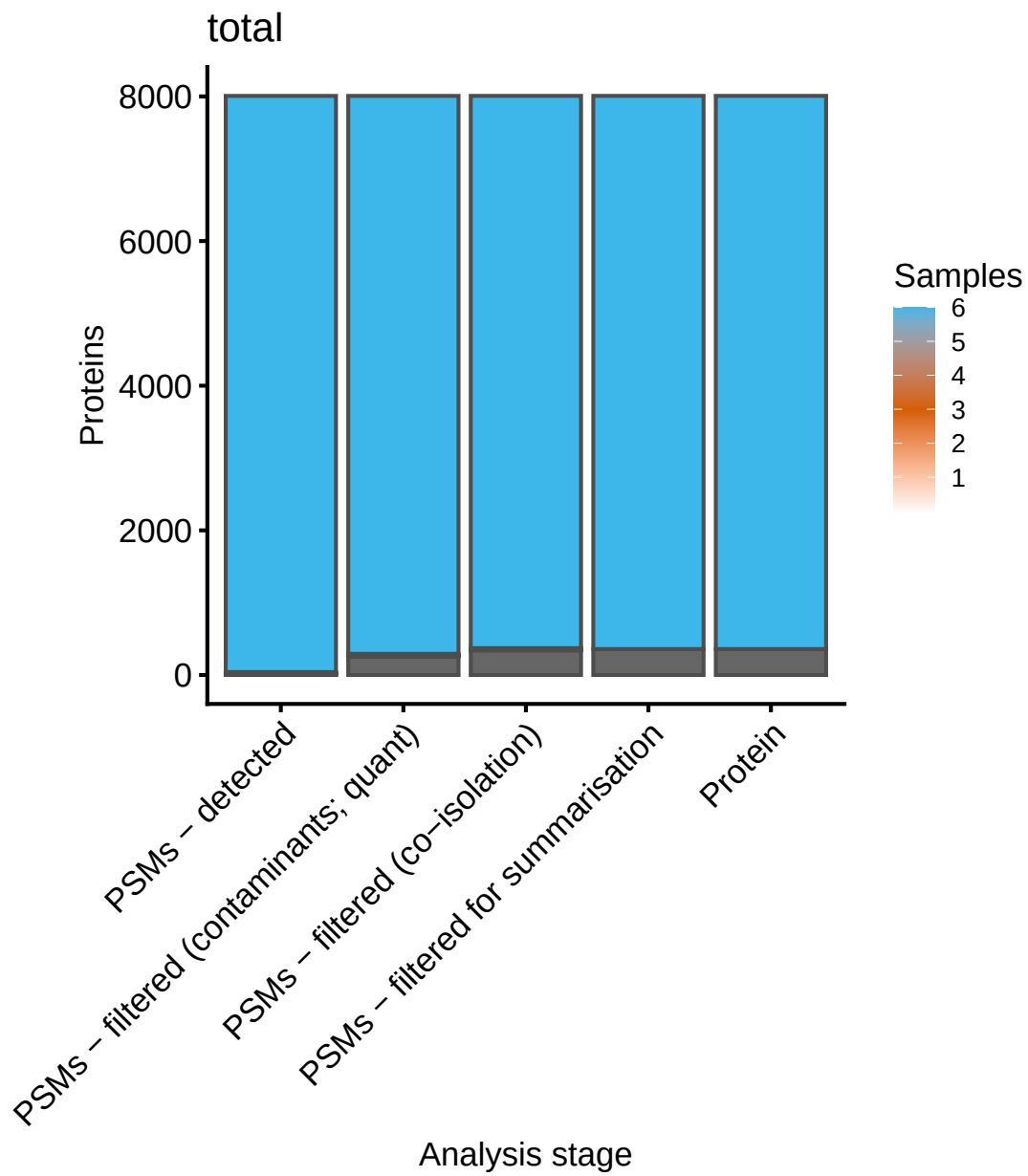
  samples_present <- get_samples_present(qfs[[qf]][, , unname(rename_cols)],
                                         rowvars, rename_cols)

  p <- plot_samples_present(samples_present, rowvars, breaks=seq(1,6,1)) +
    ylab(feature) + ggtitle(qf)

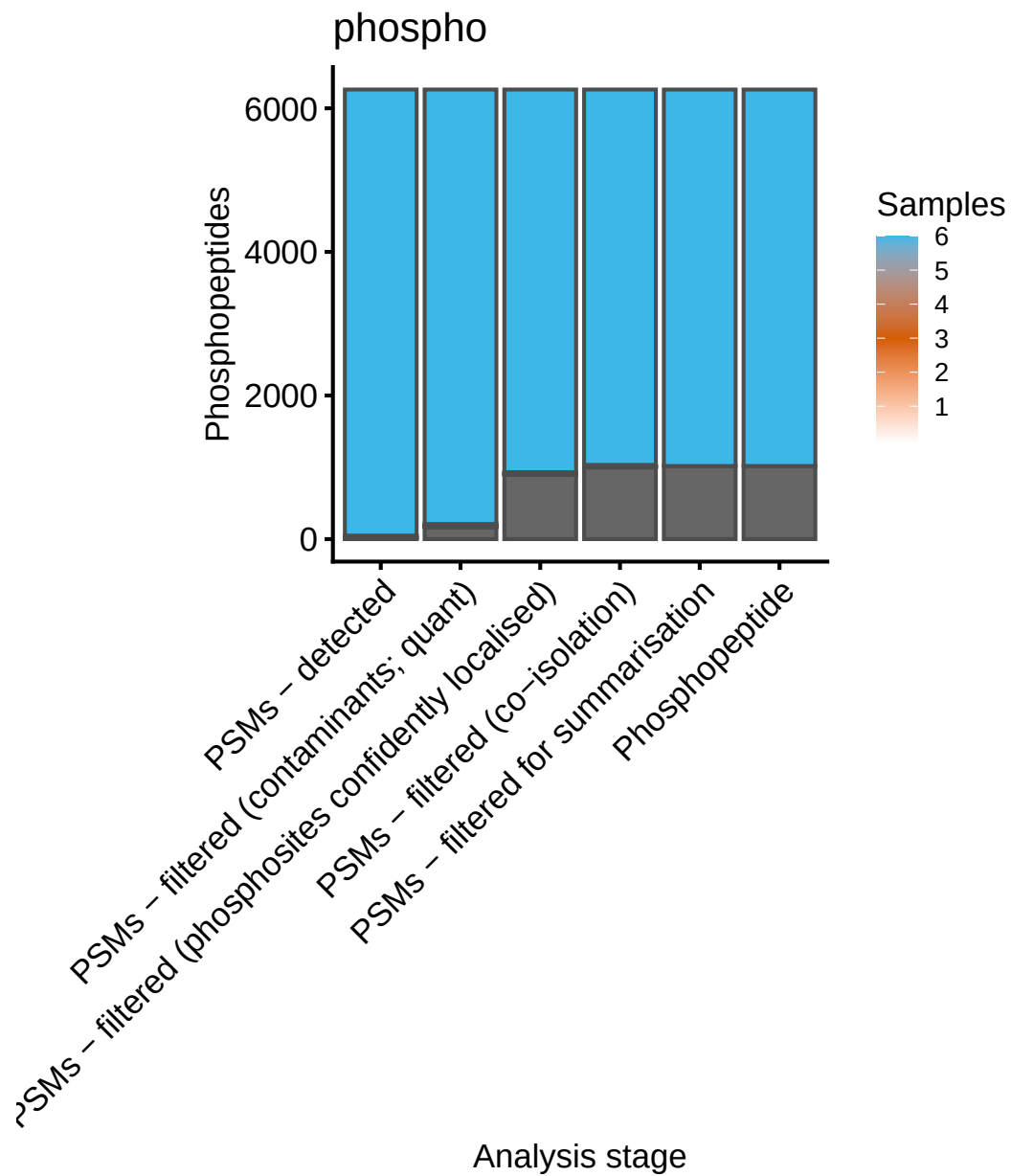
  print(p)
}

```

```
## [1] "total"
```



[1] "phospho"



7 Saving out results

Save out the QFeatures for downstream notebooks

```
output_dir <- here('notebooks/output')

saveRDS(qfs, here(output_dir, 'PROTAC_qfs.rds'))
```

Save the protein quantification in wide format

```

protein_wide_format <- assay(qfs$total[['protein']])

# Set column names to be condition_replicate
colnames(protein_wide_format) <- sprintf(
  '%s_%s', colData(qfs$total[['protein']])$condition,
  colData(qfs$total[['protein']])$replicate)

# Order columns alphabetically
protein_wide_format <- protein_wide_format[,order(colnames(protein_wide_format))]

protein_wide_format <- protein_wide_format %>%
  data.frame() %>%
  tibble::rownames_to_column('UniprotID')

write.table(protein_wide_format,
  file = here('results/PROTAC_wide_format_protein.tsv'),
  quote=FALSE, row.names=FALSE, sep='\t')

```

We will also want the quantification in ‘tidy’ (long) form in a data.frame for plotting. Below, we create and save this

```

# Given a QFeatures, with protein level abundances,
# condition and cell line colVars and
# Master.Protein.gene.name, Master.Protein.long.name and
# Master.Protein.Descriptions Rowvars, return the abundances in a long format
# data.frame
get_long_format_exp <- function(
  obj,
  colvars=c('condition', 'replicate'),
  rowvars=c('Leading.razor.protein', 'Protein.names.y',
    'Gene.Names', 'Gene.Names.First')){

  long_format_protein <- biomasslmb:::qfeatures_long(
    obj,
    colvars=colvars,
    rowvars=rowvars) %>%
    data.frame() %>%
    select(rowname,
      abundance=value,
      all_of(colvars),
      all_of(rowvars))

  return(long_format_protein)
}

long_format_protein <- get_long_format_exp(qfs$total[,,'protein']) %>%
  dplyr::rename('UniprotID'=rowname)

saveRDS(long_format_protein, here(output_dir, 'PROTAC_long_format_protein.rds'))

```

```

long_format_protein %>%
  select(UniprotID, Gene.Names, Gene.Names.First, Protein.names=Protein.names.y,
         condition, replicate, abundance) %>%
  write.table(file = here('results/PROTAC_long_format_protein.tsv'),
             quote=FALSE, row.names=FALSE, sep='\t')

long_format_phospho <- get_long_format_exp(
  qfs$phospho[,,'phosphopeptide'],
  rowvars=c('Leading.razor.protein', 'Protein.names.y', 'Gene.Names',
            'Gene.Names.First', 'ptm_positions_prot', 'prot_phos', 'ptm_name'))%>%
  dplyr::rename('phosphopeptide'=rowname)

saveRDS(long_format_phospho, here(output_dir, 'PROTAC_long_format_phospho.rds'))

long_format_phospho %>%
  select(UniprotID=Leading.razor.protein, phosphopeptide, ptm_positions_prot,
         Gene.Names, Gene.Names.First, Protein.names=Protein.names.y,
         condition, replicate, abundance) %>%
  write.table(file = here('results/PROTAC_long_format_phospho.tsv'),
             quote=FALSE, row.names=FALSE, sep='\t')

```

8 Normalising phosphoproteomics to protein abundances

Below, we create a long format data.frame containing the phosphopeptide abundances normalised to the corresponding protein abundance. This join is an inner join on UniprotID/condition/replicate: a phosphopeptide is only retained here if its parent protein also passed the total-proteome filtering above, since the normalised value requires both quantities. The normalised value (`abundance_phosphopeptide_prot_norm`) is the phosphopeptide abundance minus its parent protein abundance on the log2 scale — i.e. phosphorylation level *relative to* protein level — and is the quantity used in notebook 4's statistical testing to isolate genuine phosphorylation changes from changes that are simply a consequence of altered protein abundance.

```

long_format_phosphopeptide_norm_to_protein <- long_format_protein %>%
  select(UniprotID, condition, replicate, abundance) %>%
  merge(long_format_phospho,
        by.x=c('UniprotID', 'condition', 'replicate'),
        by.y=c('Leading.razor.protein', 'condition', 'replicate'),
        suffixes=c('.protein', '.phosphopeptide')) %>%
  mutate(
    abundance_phosphopeptide_prot_norm=abundance.phosphopeptide-abundance.protein)

saveRDS(long_format_phosphopeptide_norm_to_protein,
      here(output_dir, 'PROTAC_long_format_phosphopeptide_norm_to_protein.rds'))

long_format_phosphopeptide_norm_to_protein %>%
  select(UniprotID, phosphopeptide, ptm_positions_prot,
         Gene.Names, Gene.Names.First, Protein.names=Protein.names.y,
         condition, replicate, abundance.protein, abundance.phosphopeptide,
         abundance_phosphopeptide_prot_norm) %>%
  write.table(file = here('results/PROTAC_long_format_phosphopeptide_norm_to_protein.tsv'),
             quote=FALSE, row.names=FALSE, sep='\t')

```

```

long_long_format_phosphopeptide_norm_to_protein <- long_format_phosphopeptide_norm_to_protein %>%
  pivot_longer(cols=matches('abundance'), names_to='type', values_to='abundance') %>%
  mutate(type=gsub('abundance.', '', type)) %>%
  mutate(type=factor(type, levels=c('protein', 'phosphopeptide',
                                   'phosphopeptide_prot_norm'))))

saveRDS(long_long_format_phosphopeptide_norm_to_protein,
        here(output_dir, 'PROTAC_long_long_format_phosphopeptide_norm_to_protein.rds'))

```

9 Session info

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.1
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_GB.UTF-8/en_GB.UTF-8/en_GB.UTF-8/C/en_GB.UTF-8/en_GB.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] openxlsx_4.2.8.1      QFeatures_1.20.0
## [3] MultiAssayExperiment_1.36.2 SummarizedExperiment_1.40.0
## [5] Biobase_2.70.0        GenomicRanges_1.62.1
## [7] Seqinfo_1.0.0         IRanges_2.44.0
## [9] S4Vectors_0.48.1      BiocGenerics_0.56.0
## [11] generics_0.1.4        MatrixGenerics_1.22.0
## [13] matrixStats_1.5.0     here_1.0.2
## [15] dplyr_1.2.1           tidyr_1.3.2
## [17] ggplot2_4.0.3         biomasslmb_0.0.5
##
## loaded via a namespace (and not attached):
## [1] DBI_1.3.0              gridExtra_2.3          sandwich_3.1-1
## [4] rlang_1.2.0            magrittr_2.0.5         clue_0.3-68
## [7] cleaver_1.48.0         otel_0.2.0             compiler_4.5.2
## [10] RSQLite_3.53.2         png_0.1-9             vctrs_0.7.3
## [13] reshape2_1.4.5         stringr_1.6.0          ProtGenerics_1.42.0
## [16] pkgconfig_2.0.3        crayon_1.5.3           fastmap_1.2.0
## [19] backports_1.5.0        XVector_0.50.0         labeling_0.4.3

```

## [22] rmarkdown_2.31	visdat_0.6.0	UpSetR_1.4.1
## [25] purrr_1.2.2	bit_4.6.0	xfun_0.59
## [28] cachem_1.1.0	jsonlite_2.0.0	gmm_1.9-1
## [31] blob_1.3.0	DelayedArray_0.36.1	cluster_2.1.8.2
## [34] R6_2.6.1	stringi_1.8.7	RColorBrewer_1.1-3
## [37] genefilter_1.92.0	Rcpp_1.1.1-1.1	knitr_1.51
## [40] zoo_1.8-15	BiocBaseUtils_1.12.0	Matrix_1.7-4
## [43] splines_4.5.2	igraph_2.3.2	tidyselect_1.2.1
## [46] abind_1.4-8	yaml_2.3.12	curl_7.1.0
## [49] lattice_0.22-9	tibble_3.3.1	plyr_1.8.9
## [52] withr_3.0.3	KEGGREST_1.50.0	S7_0.2.2
## [55] tmvtnorm_1.7	evaluate_1.0.5	uniprotREST_1.0.0
## [58] survival_3.8-6	norm_1.0-11.1	zip_3.0.0
## [61] Biostrings_2.78.0	pillar_1.11.1	corrplot_0.95
## [64] checkmate_2.3.4	rprojroot_2.1.1	scales_1.4.0
## [67] xtable_1.8-8	glue_1.8.1	lazyeval_0.2.3
## [70] tools_4.5.2	robustbase_0.99-7	annotate_1.88.0
## [73] imputeLCMD_2.1	mvtnorm_1.3-3	XML_3.99-0.23
## [76] grid_4.5.2	impute_1.84.0	MsCoreUtils_1.22.1
## [79] AnnotationDbi_1.72.0	naniar_1.1.0	cli_3.6.6
## [82] S4Arrays_1.10.1	AnnotationFilter_1.34.0	pcaMethods_2.2.0
## [85] gtable_0.3.6	DEoptimR_1.2-0	digest_0.6.39
## [88] SparseArray_1.10.10	farver_2.1.2	memoise_2.0.1
## [91] htmltools_0.5.9	lifecycle_1.0.5	httr_1.4.8
## [94] bit64_4.8.2	MASS_7.3-65	